SHRI GURU TEGH BAHADUR INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY

Affiliated to [Guru Gobind Singh Indraprastha University]

SUMMER TRAINING REPORT

(Course Code: BCA 311)

on

"ML USING PYTHON"

Project: Movie Recommender System

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Computer Applications (BCA)

Submitted by:

Kunal Upadhyay

Enrollment No.: 04590202024

Course and Section: BCA 5A

Under the guidance of:

Ms Prabhjot

Session: 2026 - 2027

Department of Computer Applications

Shri Guru Tegh Bahadur Institute of Management and Information Technology

INSTITUTE'S CERTIFICATE

This is to certify that **Kunal Upadhyay**, Enrollment No. **04590202024**, a student of **Bachelor of Computer Applications (BCA)**, Section **BCA 5A**, of **Shri Guru Tegh Bahadur Institute of Management and Information Technology**, has completed the summer training (Course Code: BCA 311) at **GRAStech, Noida**, on the course **"ML Using Python"** for a period of 6 weeks, from 15 June 2026 to 31 July 2026.

During the training the student developed the project **"Movie Recommender System"** and has prepared this report on it under my guidance. The work embodied in this report is original and has not been submitted earlier for any other degree or diploma, to the best of my knowledge.

I wish the student all the success in future.

_____

Signature of the Mentor / Guide
Ms Prabhjot
Department of Computer Applications
Date: _____

_____

Signature of the Director / HOD
Shri Guru Tegh Bahadur Institute of Management and Information Technology
Date: _____

COMPANY'S CERTIFICATE



Certificate of Completion issued by GRAStech, Noida (Sr. No. GRAS/STP/26/279)

CANDIDATE DECLARATION

I, **Kunal Upadhyay**, Enrollment No. **04590202024**, a student of **Bachelor of Computer Applications (BCA)**, Section **BCA 5A**, of **Shri Guru Tegh Bahadur Institute of Management and Information Technology**, hereby declare that the summer training report entitled **"ML Using Python - Movie Recommender System"** is an authentic record of the work carried out by me during my summer training at **GRAStech, Noida**, under the guidance of **Ms Prabhjot**.

This report has been prepared by me and has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma. Wherever the work of others has been used, it has been acknowledged in the references.

_____

Kunal Upadhyay
Enrollment No.: 04590202024
Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to everyone who helped me to complete my summer training and this report.

First of all, I thank **GRAStech, Noida**, for giving me the opportunity to learn **ML Using Python** and for providing a practical learning environment. I am especially thankful to my trainer, Shubham sharma , for the clear explanations, the patient guidance and the encouragement throughout the training.

I am deeply grateful to my mentor, **Ms Prabhjot**, for her valuable guidance, regular suggestions and constant support during the preparation of this report and the project. Her feedback helped me to improve my work at every stage.

I also thank HOD and all the faculty members of the Department of Computer Applications of **Shri Guru Tegh Bahadur Institute of Management and Information Technology** for giving me the opportunity and the facilities for this training.

Finally, I thank my parents and my friends for their constant motivation and support.

Kunal Upadhyay
Bachelor of Computer Applications (BCA), BCA 5A

ABSTRACT

Recommender systems help people to find useful items from a very large number of choices. They are used by streaming platforms, online shops and music applications to save the time of the user and to increase satisfaction. This report describes the 6 weeks summer training on ML Using Python (Machine Learning using Python) completed at GRAStech, Noida, from 15 June 2026 to 31 July 2026, and the project **Movie Recommender System** developed during the training.

The system is a content-based recommender built on the TMDB 5000 Movies and Credits datasets. After the datasets were merged and the missing values removed, 4,806 movies were used. The overview, genres, keywords, cast and director of every movie were combined into a single text called tags, which was converted to lower case and stemmed with the Porter Stemmer. The tags were converted into vectors of 5,000 features with the CountVectorizer of Scikit-learn, and the cosine similarity between every pair of movies was calculated, giving a similarity matrix of size 4,806 x 4,806. The movie list and the matrix were saved with Pickle.

A web application was built with Streamlit. The user searches and selects a movie and the application shows its details and recommends the five most similar movies with their match percentage. The posters, ratings, release dates, genres and overviews are fetched from the TMDB API. Caching, timeouts and automatic retries make the application work reliably even when the network is unstable.

The report follows the phases of the software development life cycle: problem formulation, system analysis, system design, testing and implementation, and conclusion, and gives the screen snapshots and the code in the annexures.

Keywords: Machine Learning, Python, Recommender System, Content-Based Filtering, Cosine Similarity, Streamlit, TMDB API.

LIST OF FIGURES

Page No.

Figure 3.1: Overall architecture of the Movie Recommender System 10
Figure 3.2: Machine learning workflow used in the project 11
Figure 3.3: Flowchart of the recommendation process 12
Figure 3.4: Use case diagram 12
Figure 3.5: Sequence diagram of one recommendation request 13
Figure 3.6: Entity Relationship Diagram 14
Figure 3.7: Data Flow Diagram - Level 0 14
Figure 3.8: Data Flow Diagram - Level 1 15
Figure 3.9: Importing libraries and loading the datasets 16
Figure 3.10: Merging the datasets, selecting the columns and removing null values 16
Figure 3.11: Null values in the selected columns 17
Figure 3.12: Functions to convert the JSON-like columns into lists 17
Figure 3.13: Creating the tags column 18
Figure 3.14: Applying the Porter Stemmer 18
Figure 3.15: Vectorization and cosine similarity 18
Figure 3.16: Shape of the vector array 18
Figure 3.17: The recommend function in the notebook 19
Figure 3.18: Output of recommend('Gandhi') 19
Figure 3.19: Saving the model files 19
Figure 3.20: Imports and page configuration 19
Figure 3.21: CSS for the title and the button 20
Figure 3.22: Loading the model files 20
Figure 3.23: Functions to fetch movie data and poster 21
Figure 3.24: The recommend function of the application 21
Figure 3.25: Header, search box and select box 22
Figure 3.26: Displaying the selected movie 22
Figure 3.27: Displaying the recommended movies 23
Figure 6.1: Menu flow diagram of the application 35
Figure 6.2: Structure chart of app.py 35
Figure 6.3: Input screen: search box and select box 38
Figure 6.4: Input: search text typed and the filtered select box 38
Figure 6.5: Input: Get Recommendations button 38
Figure 6.6: Output: selected movie section 39
Figure 6.7: Output: five recommended movies 39
Figure 6.8: Output: More Info of a recommended movie 39
Figure 6.9: Output: warning when no movie matches 40
Figure 6.10: Output of the notebook: recommend('Gandhi') 40

LIST OF TABLES

Page No.

Table 1.1: Details of the summer training 1
Table 1.2: Topics covered during the training 1
Table 1.3: Content-based and collaborative filtering compared 3
Table 2.1: SDLC phases followed in the project 7
Table 2.2: Functional requirements 7
Table 2.3: Non-functional requirements 7
Table 2.4: Columns used from the merged dataset 8
Table 2.5: Software used and the reason for choosing it 8
Table 2.6: Hardware used 9
Table 3.1: Modules of the system 11
Table 3.2: Structure of the stored data 13
Table 3.3: Inputs of the system and their validation 15
Table 3.4: Outputs of the system 15
Table 4.1: Test data used 25
Table 4.2: Test cases and expected results 25
Table 4.3: Summary of the test report 26
Table 4.4: Problems found and how they were debugged 27
Table 4.5: Common problems and solutions 28
Table 4.6: Folder structure of the project 28
Table 5.1: Hardware requirements 31
Table 5.2: Software requirements 31
Table 6.1: Decision table of the application 36
Table 6.2: Data dictionary of movie_list.pkl 36
Table 6.3: Data dictionary of the model and the API data 36
Table 6.4: Data dictionary of important variables in app.py 37
Table 6.5: Acceptance test report 37

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
BCA	Bachelor of Computer Applications
CSS	Cascading Style Sheets
CSV	Comma Separated Values
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
ML	Machine Learning
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
PK / FK	Primary Key / Foreign Key
RAM	Random Access Memory
SDLC	Software Development Life Cycle
TC	Test Case
TF-IDF	Term Frequency - Inverse Document Frequency
TMDB	The Movie Database
UI	User Interface
URL	Uniform Resource Locator
VS Code	Visual Studio Code

CONTENTS

Page No.

CHAPTER 1: PROBLEM FORMULATION 1

1.1 Introduction about the Company 1
1.2 Introduction about the Problem 2
1.3 Present State of the Art 2
1.4 Need of Computerization 3
1.5 Proposed Software / Project 4
1.6 Importance of the Work 5

CHAPTER 2: SYSTEM ANALYSIS 6

2.1 Feasibility Study 6
2.1.1 Technical Feasibility 6
2.1.2 Economic Feasibility 6
2.1.3 Operational Feasibility 6
2.1.4 Other Feasibility Dimensions 6
2.2 Analysis Methodology 6
2.3 Choice of the Platforms 8
2.3.1 S/W used 8
2.3.2 H/W used 9

CHAPTER 3: SYSTEM DESIGN 10

3.1 Design Methodology 10
3.2 Database Design 13
3.2.1 ERD 14
3.2.2 DFD 14
3.3 Input Design 15
3.4 Output Design 15
3.5 Code Design and Development 16

CHAPTER 4: TESTING AND IMPLEMENTATION 24

4.1 Testing Methodology 24
4.1.1 Unit Testing 24
4.1.2 Module Testing 24
4.1.3 Integration Testing 24
4.1.4 System Testing 24
4.1.5 White Box / Black Box Testing 24
4.1.6 Acceptance Testing 25
4.2 Test Data & Test Cases 25
4.3 Test Reports and Debugging 26
4.4 Implementation Manual 27
4.5 Implementation 28
4.6 Users' Training 29
4.7 Post Implementation Maintenance 29

CHAPTER 5: CONCLUSION AND REFERENCES 31

5.1 Conclusion 31
5.2 System Specifications 31
5.2.1 H/W Requirement 31
5.2.2 S/W Requirement 31
5.3 Limitations of the System 32
5.4 Future Scope for Modification 32
5.5 References/Bibliography 33
CHAPTER 6: ANNEXURES 35
A-1 Menu Flow Diagram 35
A-2 Structure Chart 35
A-3 Decision Table/Tree 36
A-4 Data Dictionary 36
A-5 Test Reports 37
A-6 Sample Inputs 38
A-7 Sample Outputs 39
A-8 Coding 40

CHAPTER 1
PROBLEM FORMULATION

Learning Objectives of this Chapter

- To understand the problem that the project solves.
- To know the training organization and the training programme completed.
- To study how movie recommendation is done at present and what its limitations are.
- To understand why the work should be computerized.
- To know the objectives, scope and importance of the proposed Movie Recommender System.

CHAPTER 1: PROBLEM FORMULATION

1.1 Introduction about the Problem

Today a viewer can choose from thousands of movies through cinemas, television, and online platforms. This large choice creates a new problem: the viewer spends more time searching for a movie than watching one. Genre lists and alphabetical lists do not help much, because two movies of the same genre can be very different in story, mood and style. A person who has enjoyed one movie usually wants to find other movies that feel similar to it, but doing this manually needs a lot of knowledge about movies and a lot of time.

The problem taken in this project is: **to design and develop a web-based system which, for a movie selected by the user, automatically finds the most similar movies and shows them along with their posters and basic details.**

1.2 Introduction about the Company

I completed my summer training at **GRAStech**, Noida, on the course **"ML Using Python"** (Machine Learning using Python). The training was of **6 weeks**, from **15 June 2026 to 31 July 2026**, and GRAStech issued a Certificate of Completion (Sr. No. GRAS/STP/26/279) on 31 July 2026. The certificate carries the ISO 9001:2015 certified mark of the organization and a copy of it is attached in this report after the institute's certificate.

Table 1.1: Details of the summer training

Particular	Details
Training organization	GRAStech, Noida
Course	ML Using Python
Duration	6 weeks
Start date	15 June 2026
End date	31 July 2026
Certificate number	GRAS/STP/26/279 (issued on 31 July 2026)
Trainer at GRAStech	Mr. Shubam Sharma
Mentor from the institute	Ms Prabhjot

Project developed	Movie Recommender System

The training started with the basics of Python and moved step by step to data handling and machine learning. The topics covered are given in the following table.

Table 1.2: Topics covered during the training

S. No.	Module	Topics
1	Python Programming	Data types, operators, conditions, loops, functions, strings, lists, tuples, sets, dictionaries, file handling and exception handling.
2	Data Handling	NumPy arrays; Pandas Series and DataFrame; reading CSV files; selecting, filtering, merging and cleaning data.
3	Machine Learning Basics	Types of machine learning, features, training and testing data, model evaluation.
4	Text Processing	Cleaning of text, stemming, stop words and vectorization of text.
5	Recommender Systems	Similarity measures, cosine similarity and content-based recommendation.
6	Project Work	Development of the Movie Recommender System with a Streamlit web interface.

During the training I learned to write Python programs, to load and clean data with Pandas, to convert text into numbers, to measure similarity between items, to save a trained model with Pickle and to build a web application with Streamlit. I also learned to call an online API using the Requests library and to handle network errors. I applied all of this in the project described in this report.

1.3 Present State of the Art

At present, people choose movies in the following ways:

- By asking friends and family, or by reading reviews and "top 10" lists. These depend on personal opinion.
- By browsing genre lists, popularity lists and search boxes on websites and streaming platforms. These work only if the viewer already knows what to look for.

- By the recommendation engines of big streaming platforms. These use the watch history and ratings of millions of users (collaborative filtering), often together with the details of the movies (content-based filtering).

The recommendation engines of large platforms are very powerful but they are closed systems, they need data of a huge number of users and they cannot be used by a student or a small website. Collaborative filtering also suffers from the "cold start" problem: it cannot say anything about a new movie or a new user because no ratings exist yet. Content-based filtering does not have this problem because it needs only the details of the movies themselves. The following table compares the two main techniques with respect to this project.

Table 1.3: Content-based and collaborative filtering compared

Basis	Content-based	Collaborative
Data needed	Details of the movies (story, genre, keywords, cast, director)	Ratings or watch history of many users
New movie or new user	Works immediately	Cold start problem
Reason for a suggestion	Easy to explain (similar story, cast, director)	Difficult to explain
Complexity	Low	Medium to high
Used in this project	Yes	No

The TMDB 5000 dataset used in this project contains rich details of the movies but no information about individual users. Therefore content-based filtering was selected.

1.4 Need of Computerization

Finding similar movies manually means remembering or reading about hundreds of movies and comparing their story, cast and director. This is slow, depends on the knowledge of one person and cannot be repeated for thousands of movies. A computer can compare all 4,806 movies of the dataset with each other and find the best matches in a fraction of a second. Computerization gives the following benefits:

- **Speed:** the result appears within a few seconds after clicking the button.

- **Accuracy and consistency:** every movie is compared with the same rules and the same data, so the result does not depend on the mood or the knowledge of a person.
- **Wider coverage:** lesser-known movies, which the viewer may never have heard of, can also be recommended.
- **Rich information:** posters, ratings and overviews are fetched automatically, so the viewer can decide without opening another website.
- **Reusability:** the model is calculated once and used again and again.

1.5 Proposed Software / Project

The proposed software is a web-based **Movie Recommender System**. The user searches for a movie, selects it and clicks the Get Recommendations button. The system shows the details of the selected movie and five movies most similar to it, each with poster, rating, match percentage and an expandable More Info section.

Objectives

- To collect, merge and clean the TMDB 5000 Movies and Credits datasets.
- To convert the story, genres, keywords, cast and director of every movie into numerical vectors.
- To find the similarity between movies with cosine similarity and recommend the top five.
- To save the model in pickle files so that it can be reused without repeating the calculation.
- To build an attractive and easy web interface with Streamlit and to show live posters and details from the TMDB API.
- To test the system and make it reliable against network and data errors.

Scope

- The system works on the 4,806 movies of the TMDB 5000 dataset and gives five recommendations for one selected movie at a time.
- No registration or login is required, so anybody can use it immediately.

- Posters and details need an internet connection because they come from the TMDB API.
- The system can be extended later with more movies, more filters and other recommendation techniques.

Methodology followed

1. **Data collection:** the two CSV files were loaded with Pandas.
2. **Data preparation:** the files were merged, the required columns were selected, missing values were removed and the JSON-like text columns were converted into lists.
3. **Feature creation:** overview, genres, keywords, cast and director were joined into one text called tags, converted to lower case and stemmed.
4. **Vectorization:** the tags were converted into vectors of 5,000 features with CountVectorizer.
5. **Similarity:** the cosine similarity between all pairs of movies was calculated.
6. **Saving the model:** the movie list and the similarity matrix were saved with Pickle.
7. **Web application:** a Streamlit application was built that loads the model, recommends five movies and shows their details from TMDB.
8. **Testing and improvement:** the application was tested and the problems found were corrected.

1.6 Importance of the Work

Recommendation systems are used in almost every digital service: video and music platforms, online shopping, news and social media. They save the time of the user and increase satisfaction and engagement. Building one gave me practical experience of the complete life cycle of a machine learning application, from raw data to a working web page. The project also shows that a useful recommender can be built with free tools and a free dataset, which makes it suitable for learning, demonstration and further development. Because the result can be easily judged by any viewer (are the suggested movies really similar?), the project is also easy to present and evaluate.

CHAPTER 2
SYSTEM ANALYSIS

Learning Objectives of this Chapter

- To check whether the project is feasible technically, economically and operationally.
- To understand the methodology followed to analyze and develop the system.
- To study the requirements and the dataset of the system.
- To know the software and hardware platforms chosen and the reasons for choosing them.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Feasibility Study

Before starting the development, I checked whether the project can be completed successfully with the available time, tools and knowledge. The study was done from the following points of view.

2.1.1 Technical Feasibility

The project needs only open source tools: Python, Pandas, NumPy, NLTK, Scikit-learn, Streamlit and Requests. They are well documented, easy to install with pip and were part of the training. The dataset is freely available and the model (4,806 x 4,806 similarity matrix of about 185 MB) can be handled by an ordinary laptop with 8 GB of RAM. The project is therefore technically feasible.

2.1.2 Economic Feasibility

All the software used is free. The TMDB API is free for non-commercial use after registration, and no special hardware is needed. The cost of development and running is practically zero, so the project is economically feasible.

2.1.3 Operational Feasibility

The application has one simple page with a search box, a select box and one button. Any person who can use a web browser can operate it without training, and the results are shown with posters so they are easy to understand. The project is operationally feasible.

2.1.4 Other Feasibility Dimensions

- **Schedule feasibility:** the whole work (learning, model building, web application and testing) had to be completed within the 6 weeks of the training. The work was divided into small steps and could be completed in this time.
- **Legal feasibility:** the datasets are publicly available for learning and the TMDB API is used according to its terms for non-commercial use. TMDB should be credited as the source of the data, which is done in the footer of the application.
- **Data and privacy feasibility:** the system does not collect or store any personal data of the user, so there is no privacy risk.

2.2 Analysis Methodology

The project was developed by following the phases of the Software Development Life Cycle (SDLC) in an iterative way: after testing, the earlier phases were revisited to correct the problems found. The activities carried out in every phase are given below.

Table 2.1: SDLC phases followed in the project

Phase	Activity done in this project
Requirement analysis	Studied how movies are chosen at present, identified the problem and listed the functional and non-functional requirements.
Feasibility study	Checked technical, economic, operational, schedule and legal feasibility (section 2.1).
Design	Prepared the architecture, flowchart, data flow diagrams, use case diagram, ER diagram and screen layout (Chapter 3).
Development	Built the model in a Jupyter Notebook and the web application in Streamlit.
Testing	Tested the functions, the modules, the complete system and the requirements (Chapter 4).
Implementation	Installed the application on the computer and prepared the instructions to run it.
Maintenance	Planned how the system will be kept working and updated (section 4.7).

Functional requirements

Table 2.2: Functional requirements

ID	Requirement
FR1	The system shall load the movie list and the similarity matrix from the saved model files when it starts.
FR2	The system shall let the user type a movie name to filter the list of movies.
FR3	The system shall let the user select one movie from the matching movies.
FR4	The system shall find the five most similar movies when the user clicks Get Recommendations.
FR5	The system shall fetch the poster, rating, release date, genres and overview of the selected movie from the TMDB API.
FR6	The system shall show poster, title, rating, match percentage and more information for every recommended movie.
FR7	The system shall show a clear message if no movie matches the search text.
FR8	The system shall continue to work and show a message if a poster or details cannot be fetched.

Non-functional requirements

Table 2.3: Non-functional requirements

Requirement	Description
Performance	Recommendations should appear within a few seconds; the model and the API responses are cached.
Usability	Simple page with a clear search box, select box and button.
Reliability	A temporary network failure must not crash the application; failed API calls are retried.
Maintainability	Code is divided into small functions with clear names.
Portability	Runs on any operating system that supports Python and Streamlit.
Security	No personal data is stored; the API key should be kept in an environment variable before sharing the code.

Analysis of the data

The TMDB 5000 Movies dataset has 4,803 movies and 20 columns, and the TMDB 5000 Credits dataset gives the cast and crew of the same movies. After the two files were merged on the title the data had 4,809 rows and 23 columns (some titles are repeated, so there are more rows than movies). Only seven columns describe the content of a movie and were kept; columns such as budget, revenue, popularity and homepage were dropped. Three rows had no overview and were removed, which left 4,806 movies.

Table 2.4: Columns used from the merged dataset

Column	Use in the project
movie_id	TMDB ID of the movie, used to fetch poster and details from the API
title	Name of the movie, shown in the select box
overview	Story summary, used to create tags
genres	Genres of the movie, used to create tags
keywords	Keywords describing the theme, used to create tags
cast	Actors of the movie, used to create tags
crew	Only the director is taken from the crew, used to create tags

2.3 Choice of the Platforms

2.3.1 S/W used

Table 2.5: Software used and the reason for choosing it

Software	Use in this project
Python 3.13	Single language for both the machine learning part and the web part.
Jupyter Notebook (in VS Code)	Step by step development of the model; the output of every step can be seen immediately.
Pandas and NumPy	Reading, merging and cleaning the data; storing the vectors and the similarity matrix.
NLTK (PorterStemmer)	Reducing words to their root form, so that loved, loving and loves count as one word.
Scikit-learn	CountVectorizer to create the 5,000 features and cosine_similarity to compare the movies.
Pickle	Saving the movie list and the similarity matrix, so the heavy calculation is done only once.
Streamlit	Building the web page with very little code; caching decorators avoid repeated loading.
Requests	Sending requests to the TMDB API.
TMDB API	Source of posters, ratings, release dates, genres and overviews.
Visual Studio Code	Editor, notebook environment and terminal.

2.3.2 H/W used

Table 2.6: Hardware used

Component	Specification
Computer	Laptop with Windows
Processor	[Processor of your laptop, e.g. Intel Core i5]
RAM	[e.g. 8 GB]
Storage	At least 1 GB free space for the dataset, model and code
Network	Internet connection (needed for the TMDB API)

CHAPTER 3
SYSTEM DESIGN

Learning Objectives of this Chapter

- To understand the architecture and the modules of the Movie Recommender System.
- To study the flow of data with flowchart, DFD, use case and sequence diagrams.
- To study the design of the stored data, the inputs and the outputs.
- To understand how the model and the web application were coded.

CHAPTER 3: SYSTEM DESIGN

3.1 Design Methodology

The system was designed with the **top-down modular approach**. The complete system was first divided into two parts: an offline part that builds the model in a notebook, and an online part, the web application, that uses the model. Each part was then divided into small modules with one job each, so that every module could be coded and tested separately.

System architecture

Figure 3.1 shows the architecture. The user works in a web browser with the Streamlit application. The application calls the recommendation function, which reads the saved model files and returns the five most similar movies. For each movie to be displayed, the application sends a request to the TMDB API and receives the poster path, rating and other details in JSON format.

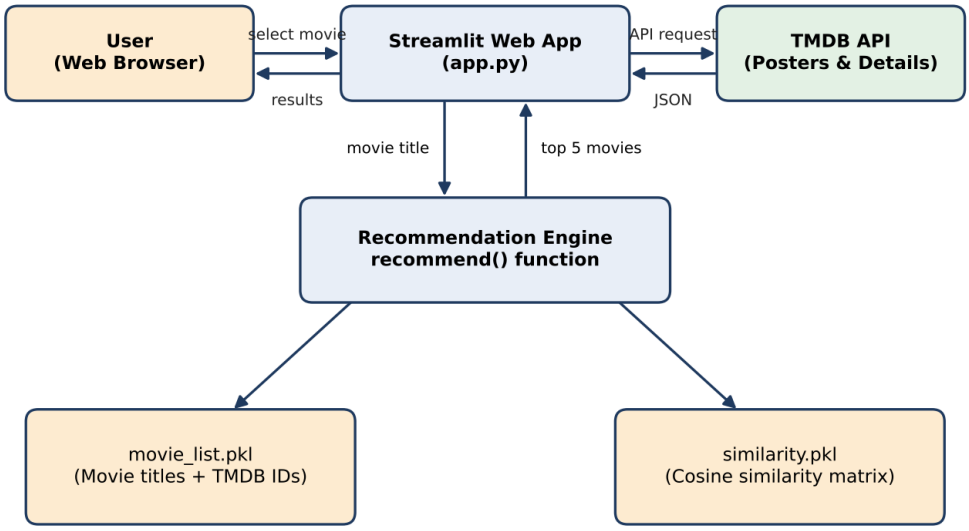


Figure 3.1: Overall architecture of the Movie Recommender System

Machine learning workflow

The model files used by the application are prepared in advance by the workflow in Figure 3.2. It is executed only once, in the Jupyter Notebook.

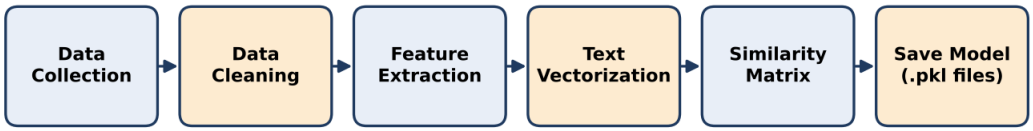


Figure 3.2: Machine learning workflow used in the project

Modules of the system

Table 3.1: Modules of the system

Module	Job of the module
Data preparation (notebook)	Loads and merges the datasets, removes null values and converts the JSON-like columns into lists.
Feature creation (notebook)	Creates the tags column, converts it to lower case and applies stemming.
Model building (notebook)	Vectorizes the tags, calculates the cosine similarity and saves both files with Pickle.
load_model() (app.py)	Loads the movie list and the similarity matrix when the application starts.
Search and select (app.py)	Filters the movie list by the text typed and lets the user select a movie.
recommend() (app.py)	Finds the five movies with the highest similarity score.
get_movie_data(), fetch_details(), get_poster() (app.py)	Fetch movie details and poster from TMDB with retry and caching.
Display (app.py)	Shows the selected movie and the recommended movies as cards.

Flowchart

Figure 3.3 shows the working of the web application. When the Get Recommendations button is clicked, the index of the selected movie is found, all movies are sorted by their similarity score, the top five are taken and their posters and details are fetched and displayed.

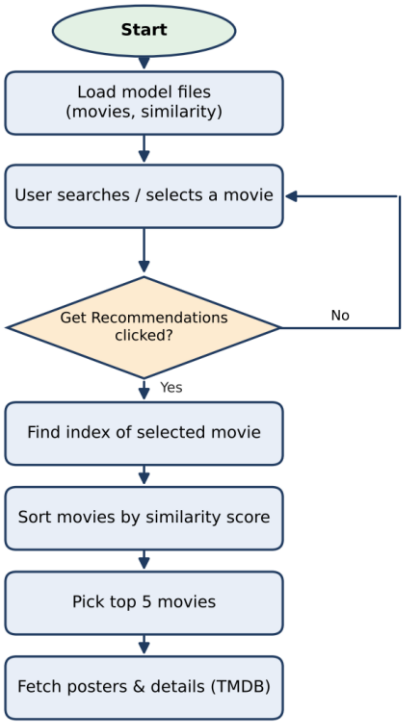


Figure 3.3: Flowchart of the recommendation process

Use case diagram

Figure 3.4 shows what the user can do with the system. The user is the only actor.

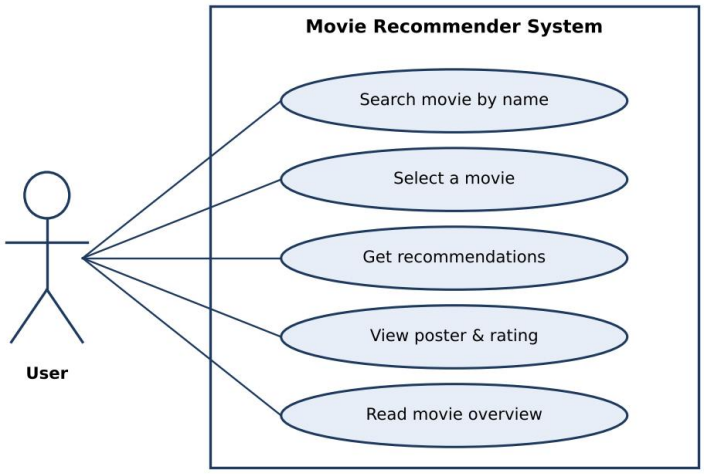


Figure 3.4: Use case diagram

Sequence diagram

Figure 3.5 shows the order of messages between the user, the Streamlit interface, the recommendation engine and the TMDB API for one recommendation request.

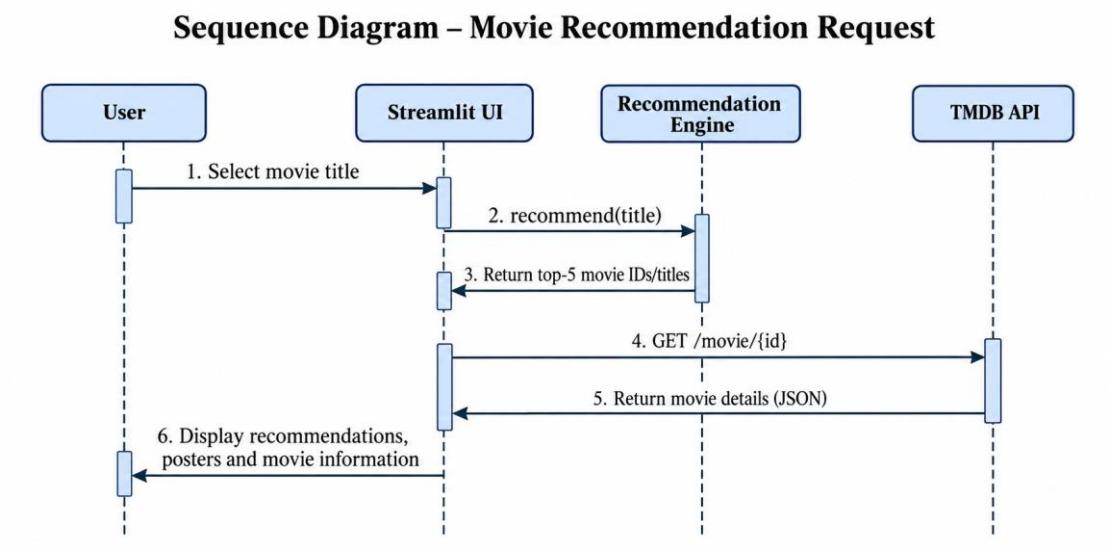


Figure 3.5: Sequence diagram of one recommendation request

3.2 Database Design

The system does not use a database server. All the data of the model is kept in two pickle files, and the details of movies are fetched from TMDB when needed. The data is, however, designed as tables with keys, and the usual rules of normalization were followed so that no data is repeated and every value has one place.

Table 3.2: Structure of the stored data

File / Source	Structure	Key
movie_list.pkl	DataFrame of 4,806 rows with the columns movie_id, title and tags	movie_id
similarity.pkl	NumPy array of size 4,806 x 4,806; the value at row i and column j is the similarity of movie i and movie j	Row number, column number
TMDB API response	JSON with poster_path, vote_average, release_date, genres, overview and other fields	movie_id

Normalization and validation

- **First normal form:** every column of movie_list holds one value (the id, one title, one text of tags); the repeating groups such as the list of genres or actors were joined into the single tags text.
- **Second and third normal form:** title and tags depend only on movie_id and not on each other, and the details that belong to a movie (poster, rating, release date) are not copied into the model but fetched using movie_id.

- **Validation:** rows without overview were removed; duplicate rows were checked (none found); the row number of a movie in movie_list is always equal to its row number in the similarity matrix (reset_index is applied after loading).
- **Verification:** the shape of the vector array (4806, 5000) was checked after vectorization, and the recommend() function was tested with the movie Gandhi.

3.2.1 ERD

Figure 3.6 shows the entities of the system and their relations. One MOVIE has exactly one set of TMDB_DETAILS, and one MOVIE appears in many rows of the SIMILARITY matrix (its similarity with every other movie).

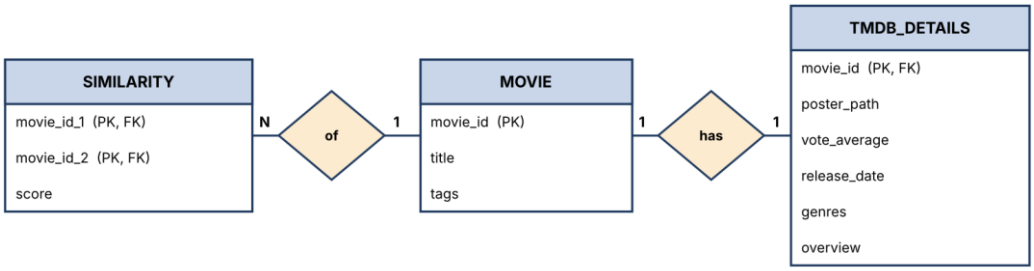


Figure 3.6: Entity Relationship Diagram

3.2.2 DFD

Figure 3.7 is the context diagram (Level 0). The user gives a movie name and receives recommendations; the system sends a movie ID to TMDB and receives the poster and details.

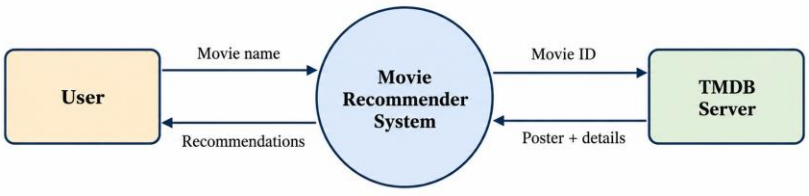


Figure 3.7: Data Flow Diagram - Level 0

Figure 3.7: Data Flow Diagram - Level 0

Figure 3.8 divides the system into three processes: searching and selecting a movie (1.0), generating recommendations (2.0) and fetching poster and details (3.0). Process 2.0 reads the data store D1 (the pickle files) and process 3.0 talks to the TMDB API.

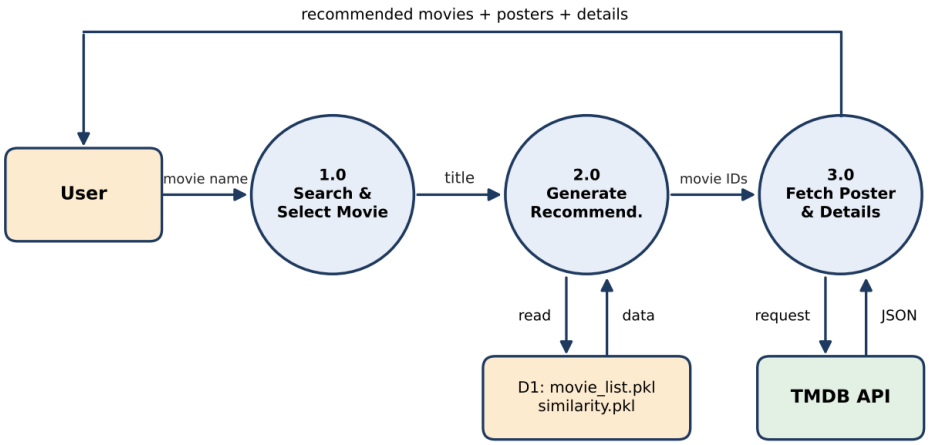


Figure 3.8: Data Flow Diagram - Level 1

3.3 Input Design

The system takes very few inputs, so the input screen is kept simple. Every input has a check so that a wrong input cannot break the system.

Table 3.3: Inputs of the system and their validation

Input	Control used	Validation
Search text	Text box	Leading and trailing spaces are removed; the search is not case-sensitive; special characters are treated as plain text, not as a pattern; an empty text shows all movies.
Selected movie	Select box	Only the titles of the movies present in the dataset can be selected; duplicate titles are removed from the list.
Get Recommendations	Button	Recommendations are generated only after the button is clicked.

If no movie matches the search text, a warning is shown and the page stops instead of showing an error. Screen snapshots of the inputs are given in Annexure A-6.

3.4 Output Design

The outputs are designed to be easy to read and to look like a movie application. Posters are shown because they help the viewer to recognize and judge a movie quickly.

Table 3.4: Outputs of the system

Output	Contents	Format
Selected movie section	Poster, name, rating, release date, genres and overview of the selected movie	Poster in a narrow column, details in a wide column
Recommended movies	Five cards, each with poster, title, rating and match percentage	Cards with border, five in one row
More Info	Release date and overview of a recommended movie	Expandable section inside the card
Messages	No movie found, poster unavailable, details unavailable	Warning, information and caption messages

The page has a gradient title, a search box with a select box in one row, a red Get Recommendations button, the selected movie section, five recommended movie cards and a footer. Screen snapshots of the outputs are given in Annexure A-7.

3.5 Code Design and Development

The code is written in simple, small functions with meaningful names. The development was done in two parts: the model in a Jupyter Notebook and the web application in a Python file, app.py. The code of every step is shown below.

Part A: Building the model in the notebook

Step 1 - Loading the datasets. The two CSV files are read into the DataFrames movies and credits.

```
1 import numpy as np
2 import pandas as pd
3
4 import pandas as pd
5
6 movies = pd.read_csv('../Dataset/tmdb_5000_movies.csv')
7 credits = pd.read_csv('../Dataset/tmdb_5000_credits.csv')
```

Figure 3.9: Importing libraries and loading the datasets

Step 2 - Merging and selecting columns. The two files are merged on the title. The merged data has 4,809 rows and 23 columns. Only seven columns are kept. Three rows with a missing overview are removed.

```
movies = movies.merge(credits,on='title')

movies = movies[['movie_id','title','overview','genres','keywords','cast','crew']]

movies.dropna(inplace=True)

movies.duplicated().sum()
```

Figure 3.10: Merging the datasets, selecting the columns and removing null values

```
movie_id    0
title       0
overview    3
genres      0
keywords    0
cast        0
crew        0
dtype: int64
```

Figure 3.11: Null values in the selected columns

Step 3 - Converting the text columns into lists. The genres, keywords, cast and crew columns are stored as JSON-like text. The function ast.literal_eval() converts the text into a Python list and convert() collects only the names. For the crew column, fetch_director() keeps only the director.

```
def convert(text):
    L = []
    for i in ast.literal_eval(text):
        L.append(i['name'])
    return L

movies['genres'] = movies['genres'].apply(convert)

movies['keywords'] = movies['keywords'].apply(convert)
movies.head(1)

def convert3(text):
    L = []
    counter = 0
    for i in ast.literal_eval(text):
        if counter < 3:
            L.append(i['name'])
        counter+=1
    return L

movies['cast'] = movies['cast'].apply(convert)

def fetch_director(text):
    L = []
    for i in ast.literal_eval(text):
        if i['job'] == 'Director':
            L.append(i['name'])
    return L

movies['crew'] = movies['crew'].apply(fetch_director)
```

Figure 3.12: Functions to convert the JSON-like columns into lists

Step 4 - Creating the tags. Spaces inside names are removed (for example SamWorthington), so that every name is one feature. The overview is split into words

and the five columns are joined to form the tags column, which is then converted to lower case.

```
movies['overview'] = movies['overview'].apply(lambda x:x.split())

def collapse(L):
    L1 = []
    for i in L:
        L1.append(i.replace(" ",""))
    return L1

movies['cast'] = movies['cast'].apply(collapse)
movies['crew'] = movies['crew'].apply(collapse)
movies['genres'] = movies['genres'].apply(collapse)
movies['keywords'] = movies['keywords'].apply(collapse)

movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] + movies['cast'] + movies['crew']

new_df = movies.drop(columns=['overview','genres','keywords','cast','crew'])

new_df['tags'] = new_df['tags'].apply(lambda x: " ".join(x))

new_df['tags'] = new_df['tags'].apply(lambda x: x.lower())
```

Figure 3.13: Creating the tags column

Step 5 - Stemming. The Porter Stemmer reduces every word to its root form.

```
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()

def stem(text):
    y = []
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)

new_df['tags'] = new_df['tags'].apply(stem)
```

Figure 3.14: Applying the Porter Stemmer

Step 6 - Vectorization and similarity. CountVectorizer keeps the 5,000 most frequent words and ignores English stop words. The array has the shape (4806, 5000). The cosine similarity of all pairs is then calculated and stored in similarity.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000,stop_words='english')

vector = cv.fit_transform(new_df['tags']).toarray()

from sklearn.metrics.pairwise import cosine_similarity

similarity = cosine_similarity(vector)
```

Figure 3.15: Vectorization and cosine similarity

```
(4806, 5000)
```

Figure 3.16: Shape of the vector array

Step 7 - The recommend function. The function finds the index of the movie, sorts the similarity scores in descending order, skips the movie itself and prints the next five. It was tested with the movie Gandhi.

```python
def recommend(movie):
    index = new_df[new_df['title'] == movie].index[0]
    distances = sorted(list(enumerate(similarity[index])),reverse=True,key = lambda x: x[1])
    for i in distances[1:6]:
        print(new_df.iloc[i[0]].title)

recommend('Gandhi')
```

Figure 3.17: The recommend function in the notebook

```
Ramanujan
Guiana 1838
The Wind That Shakes the Barley
The Bounty
A Passage to India
```

Figure 3.18: Output of recommend('Gandhi')

The five movies suggested for Gandhi (Ramanujan, Guiana 1838, The Wind That Shakes the Barley, The Bounty and A Passage to India) are historical dramas connected with the British Empire and its colonies, which is also the background of Gandhi. This shows that the tags capture the theme of a movie well.

Step 8 - Saving the model. The movie list and the similarity matrix are saved with pickle.dump().

```python
import pickle

pickle.dump(new_df,open('movie_list.pkl','wb'))
pickle.dump(similarity,open('similarity.pkl','wb'))
```

Figure 3.19: Saving the model files

Part B: Developing the web application (app.py)

Step 1 - Imports and page settings. The wide layout is used so that five cards fit in one row.

```python
import os
import time
import pickle
import requests
import streamlit as st

st.set_page_config(
    page_title="Movie Recommender System",
    page_icon="",
    layout="wide"
)
```

Figure 3.20: Imports and page configuration

Step 2 - Styling. A small CSS gives the title a red to gold gradient and the button a red colour.

```
st.markdown("""
<style>
.title {
    text-align: center;
    font-size: 3rem;
    font-weight: 800;
    background: linear-gradient(90deg, #FF4B4B, #FFD700);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
.subtitle {
    text-align: center;
    color: gray;
    margin-bottom: 20px;
}
.stButton > button {
    background-color: #FF4B4B;
    color: white;
    border-radius: 8px;
    font-weight: bold;
    border: none;
}
.stButton > button:hover {
    background-color: #E63946;
    color: white;
}
</style>
""", unsafe_allow_html=True)
```

Figure 3.21: CSS for the title and the button

Step 3 - Loading the model. load_model() reads the two pickle files and is cached with @st.cache_resource, so the large matrix is loaded only once. If a file is missing, a message is shown and the application stops. The line reset_index(drop=True) makes the row label of every movie equal to its position, which is needed because three rows were removed in the notebook.

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_DIR = os.path.join(os.path.dirname(BASE_DIR), "Model")


@st.cache_resource
def load_model():
    movies = pickle.load(open(os.path.join(MODEL_DIR, "movie_list.pkl"), "rb"))
    similarity = pickle.load(open(os.path.join(MODEL_DIR, "similarity.pkl"), "rb"))
    return movies, similarity


try:
    movies, similarity = load_model()
except Exception as error:
    st.error(f"Model load nahi hua: {error}")
    st.info("Check karo ki movie_list.pkl aur similarity.pkl Model folder mein hain.")
    st.stop()

movies = movies.reset_index(drop=True)
```

Figure 3.22: Loading the model files

Step 4 - Fetching data from TMDB. get_movie_data() sends the request with a timeout of 15 seconds and tries up to three times if the connection fails. fetch_details() returns None instead of crashing when all attempts fail, and get_poster() builds the full poster address. Results are cached for one hour.

```python
API_KEY = "YOUR_TMDB_API_KEY_HERE"
BASE_URL = "https://api.themoviedb.org/3"


@st.cache_data(ttl=3600, show_spinner=False)
def get_movie_data(movie_id):
    for attempt in range(3):
        try:
            response = requests.get(
                f"{BASE_URL}/movie/{int(movie_id)}",
                params={"api_key": API_KEY, "language": "en-US"},
                timeout=15
            )
            response.raise_for_status()
            return response.json()
        except requests.RequestException as error:
            print(f"TMDB error (try {attempt + 1}) for movie {movie_id}: {error}")
            time.sleep(1)

    raise ConnectionError("TMDB se data nahi mil paaya")


def fetch_details(movie_id):
    try:
        return get_movie_data(movie_id)
    except Exception:
        return None


def get_poster(details):
    if details and details.get("poster_path"):
        return "https://image.tmdb.org/t/p/w500" + details["poster_path"]
    return None
```

Figure 3.23: Functions to fetch movie data and poster

Step 5 - Recommendation function. It returns a list of five movies with title, movie_id and similarity score.

```python
def recommend(movie_title):
    movie_index = movies[movies["title"] == movie_title].index[0]

    distances = sorted(
        list(enumerate(similarity[movie_index])),
        key=lambda x: x[1],
        reverse=True
    )

    result = []
    for index, score in distances[1:6]:
        result.append({
            "title": movies.iloc[index]["title"],
            "movie_id": movies.iloc[index]["movie_id"],
            "score": score
        })
    return result
```

Figure 3.24: The recommend function of the application

Step 6 - Header, search and select. The movies are filtered by the text typed and the duplicates are removed from the list.

```python
st.markdown("<div class='title'>Movie Recommender System</div>", unsafe_allow_html=True)
st.markdown("<div class='subtitle'>Discover your next favorite movie!</div>", unsafe_allow_html=True)

search_col, select_col = st.columns([1, 2])

with search_col:
    search_text = st.text_input("Search for a movie", placeholder="Enter movie name...")

if search_text.strip():
    filtered = movies[movies["title"].str.contains(search_text.strip(), case=False, na=False, regex=False)]
else:
    filtered = movies

movie_list = filtered["title"].dropna().unique().tolist()

if len(movie_list) == 0:
    st.warning("Koi movie nahi mili. Dusra naam search karo.")
    st.stop()

with select_col:
    selected_movie = st.selectbox("Select a movie", movie_list)
```

Figure 3.25: Header, search box and select box

Step 7 - Selected movie. On clicking the button the poster, rating, release date, genres and overview of the selected movie are shown.

```python
if st.button("Get Recommendations"):
    movie_id = movies[movies["title"] == selected_movie].iloc[0]["movie_id"]
    details = fetch_details(movie_id)

    st.subheader("Selected Movie")
    col1, col2 = st.columns([1, 3])

    with col1:
        poster = get_poster(details)
        if poster:
            st.image(poster)
        else:
            st.info("Poster available nahi hai")

    with col2:
        st.markdown(f"## {selected_movie}")
        if details:
            genres = ", ".join([g["name"] for g in details.get("genres", [])])
            st.write(f" **Rating:** {details.get('vote_average', 'N/A')}")
            st.write(f"**Release Date:** {details.get('release_date', 'N/A')}")
            st.write(f"**Genres:** {genres}")
            st.write(details.get("overview", "No overview available."))
        else:
            st.info("Details fetch nahi ho paayi. Internet ya API key check karo.")
```

Figure 3.26: Displaying the selected movie

Step 8 - Recommended movies. Five bordered cards are shown, each with poster, title, rating, match percentage and a More Info section.

```
    st.divider()

    st.subheader("Recommended Movies")

    recommendations = recommend(selected_movie)
    columns = st.columns(5)

    for column, movie in zip(columns, recommendations):
        with column:
            with st.container(border=True):
                movie_details = fetch_details(movie["movie_id"])

                poster = get_poster(movie_details)
                if poster:
                    st.image(poster)
                else:
                    st.caption("Poster unavailable")

                st.markdown(f"**{movie['title']}**")

                if movie_details:
                    st.write(f" {movie_details.get('vote_average', 'N/A')}")
                st.caption(f"Match: {round(movie['score'] * 100)}%")

                if movie_details:
                    with st.expander("More Info"):
                        st.write("**Release Date:**", movie_details.get("release_date", "N/A"))
                        st.write(movie_details.get("overview", "No overview available."))

st.divider()
st.caption("Made with using Streamlit & TMDB API")
```

Figure 3.27: Displaying the recommended movies

CHAPTER 4

TESTING AND IMPLEMENTATION

Learning Objectives of this Chapter

- To understand the testing approach followed at the different levels.
- To study the test data, test cases and test report of the system.
- To see how the errors found in testing were debugged.
- To know how the system is installed, used and maintained.

CHAPTER 4: TESTING AND IMPLEMENTATION

4.1 Testing Methodology

Testing was done at several levels, from single functions to the complete system, and finished with an acceptance test based on the requirements (section 2.2) and the user manual (section 4.6). Both the notebook and the web application were tested. The testing was done by running the code with sample inputs and comparing the actual result with the expected result.

4.1.1 Unit Testing

Every function was tested separately with sample inputs. convert() and fetch_director() were run on the first movie (Avatar) and their output was compared with the original data; stem() was run on a short sentence; recommend() was run for a known movie (Gandhi); and get_poster() was run with a valid and an invalid poster path.

4.1.2 Module Testing

Each module of Table 3.1 was tested as a whole: the data preparation module (row counts, null values and duplicates were checked after every step), the model building module (the shape of the vector array and of the similarity matrix were checked), the TMDB module (with a valid movie ID, an invalid ID and with the internet switched off) and the display module.

4.1.3 Integration Testing

After the modules worked, they were joined and tested together: the pickle files created in the notebook were loaded by the application, the list returned by recommend() was passed to the TMDB functions and the result was displayed. It was checked that the movie_id, the title and the similarity row stay consistent along the whole path.

4.1.4 System Testing

The complete application was run with streamlit run app.py and operated like a normal user: searching, selecting, clicking the button and opening More Info. Performance, error messages and behaviour without internet were also checked.

4.1.5 White Box / Black Box Testing

- **White box testing** was done with the knowledge of the code: every branch of the main functions was exercised, for example the success path and the failure

path of get_movie_data(), the case where details are None, the case where the poster path is missing and the case where the filtered list is empty.

- **Black box testing** was done without looking at the code, only by giving inputs and checking the outputs: valid movie names, upper and lower case search text, text with no match, special characters, and no internet.

4.1.6 Acceptance Testing

Finally each functional requirement FR1 to FR8 was checked against the working application. The system is accepted when all of them are satisfied. The result is recorded in Annexure A-5.

4.2 Test Data & Test Cases

Test data

Table 4.1: Test data used

Purpose	Data
Valid movie names	Avatar, Gandhi, The Lego Movie
Search text, lower and upper case	avatar, AVATAR
Search text with no match	zzzzzz
Special characters	(and + and ?
Empty search	(nothing typed)
Invalid movie ID for API test	0 and 99999999
Network condition	Internet on, internet off

Test cases

Table 4.2: Test cases and expected results

ID	Test case	Input / action	Expected result	Status
TC1	Application start	Run streamlit run app.py	Page opens with title, search box, select box and button.	[Pass]
TC2	Model loading	Start with both pickle files present	Model loads without error.	[Pass]
TC3	Missing model file	Rename similarity.pkl and start	Message with the missing file name; the application stops.	[Pass]
TC4	Valid search	Type avatar	Select box lists only movies containing the text.	[Pass]

ID	Test case	Input / action	Expected result	Status
TC5	Case-insensitive search	Type AVATAR	Same movies as TC4.	[Pass]
TC6	No match	Type zzzzzz	Warning that no movie was found.	[Pass]
TC7	Special characters	Type (	No error; the result is treated as plain text.	[Pass]
TC8	Get recommendations	Select Avatar and click the button	Selected movie details and five recommended movies.	[Pass]
TC9	Own movie excluded	Check the recommended list	Selected movie is not in its own list.	[Pass]
TC10	Posters	Select a movie with poster on TMDB	Posters of all six movies are shown.	[Pass]
TC11	Match percentage	Check every card	Value between 0 and 100.	[Pass]
TC12	More Info	Open More Info on a card	Release date and overview are shown.	[Pass]
TC13	No internet	Disconnect and click the button	No crash; Poster unavailable and Details unavailable are shown.	[Pass]
TC14	Caching	Select the same movie twice	Second result appears faster.	[Pass]
TC15	Notebook function	recommend('Gandhi') in the notebook	Five movies are printed (Chapter 3, Step 7).	Pass

4.3 Test Reports and Debugging

Test report

Table 4.3: Summary of the test report

Item	Result
Total test cases	15
Test cases passed	[15]
Test cases failed	[0]
Tested on	15-09-2026
Tested by	Kunal Upadhyay
Browser	Google Chrome
Python version	3.13

Debugging

During development and testing the following problems were identified. For every problem the cause was found from the error message or the behaviour of the application and the code was corrected. After each correction the related test cases were run again to be sure that nothing else was broken.

Table 4.4: Problems found and how they were debugged

Problem	Cause	Correction
Error ConnectionResetError 10054 while fetching movie details	The TMDB server closed the connection suddenly, which can happen on an unstable network when many requests are sent one after another.	A timeout was added and the request is retried up to three times with a short wait.
Poster and details missing for an hour after one failed request	The failed result (None) was stored by the cache and reused until the cache expired.	The cached function raises an error on failure and a second function handles it, because Streamlit does not cache errors.
Warning: use_container_width will be removed after 2025-12-31	The parameter is deprecated in the newer Streamlit versions.	The parameter was removed from the st.image() calls.
Wrong row of the similarity matrix could be used for some movies	After dropna() the row labels of the DataFrame had gaps, so a label no longer matched the row position in the matrix.	reset_index(drop=True) is applied after loading the movie list.
Page showed an error when the search had no match	The select box received an empty list.	A check shows a warning and stops the page.
API key visible in the source code	The key was written directly in the program.	The key is kept in one variable and must be moved to an environment variable before the code is shared.

4.4 Implementation Manual

This manual explains how to set up and run the system on a new computer.

Requirements

- Python 3.10 or later, and Visual Studio Code (optional).
- A free TMDB account and its API key.
- The folders Dataset, Model and Website are arranged as shown in Table 4.6.

Steps

1. Open the terminal in the project folder and install the libraries: `pip install streamlit pandas numpy scikit-learn nltk requests`
2. Open the notebook Movie Recommender System.ipynb and run all the cells from top to bottom. This creates movie_list.pkl and similarity.pkl. Keep both files in the Model folder.
3. Open app.py in the Website folder and write your TMDB API key in the variable API_KEY.
4. Open the terminal in the Website folder and run: `python -m streamlit run app.py`
5. The application opens in the browser at http://localhost:8501.

Common problems

Table 4.5: Common problems and solutions

Problem	Solution
Message: Model file nahi mili	Check that both pickle files are in the Model folder, which must be next to the Website folder.
Posters or details not shown	Check the internet connection and the TMDB API key written in app.py.
ModuleNotFoundError	Install the missing library with pip install library-name.
Port already in use	Close the old Streamlit window or run with another port: streamlit run app.py --server.port 8502

4.5 Implementation

The system was implemented on a Windows laptop. First the model was built once in the notebook and the two pickle files were saved. Then the web application was run from the Website folder with Streamlit, which starts a local web server and opens the application in the browser. The files of the project are arranged in separate folders as follows.

Table 4.6: Folder structure of the project

Folder / file	Description
Dataset / tmdb_5000_movies.csv	Movies dataset
Dataset / tmdb_5000_credits.csv	Cast and crew dataset
Notebook / Movie Recommender System.ipynb	Notebook in which the model is built
Model / movie_list.pkl	Saved movie list
Model / similarity.pkl	Saved similarity matrix
Website / app.py	Streamlit web application
Website / requirements.txt	Libraries required by the application
Website / README.md	Description of the project

The system can be used by other people also if it is deployed on a cloud service such as Streamlit Community Cloud. In that case the pickle files must be uploaded with the code and the API key must be stored as a secret and not in the code.

4.6 Users' Training

The system is simple, so a short demonstration of a few minutes is enough for a new user. The user manual is given below.

1. Open the application in the browser.
2. Type a part of a movie name in the search box (optional). The select box then shows only the matching movies.
3. Choose a movie from the select box.
4. Click the **Get Recommendations** button.
5. Read the details of the selected movie: poster, rating, release date, genres and overview.
6. See the five recommended movies. The match percentage tells how similar a movie is to the selected one.
7. Click **More Info** below a card to read the release date and the overview.
8. To try another movie, change the selection and click the button again.

If a poster or details are not shown, the internet connection or the API key should be checked; the recommendations are still shown.

4.7 Post Implementation Maintenance

After the system is in use, it needs the following maintenance.

- **Corrective maintenance:** correcting errors that are reported by the users, for example a wrong poster or a message that is not clear.
- **Adaptive maintenance:** changing the code when the environment changes, for example when a Streamlit function is deprecated (as happened with use_container_width) or when the TMDB API changes.
- **Perfective maintenance:** adding features such as genre filters, more movies or trailers.
- **Data maintenance:** running the notebook again to create new pickle files when the dataset is updated; taking a backup of the pickle files.
- **Key maintenance:** renewing the TMDB API key if it is lost or leaked and keeping it outside the source code.
- **Library maintenance:** keeping the libraries listed in requirements.txt up to date and testing the application again after updating.

CHAPTER 5
CONCLUSION AND REFERENCES

Learning Objectives of this Chapter

- To summarize the work done and the results obtained.
- To list the hardware and software requirements of the system.
- To know the limitations of the system and the scope for improvement.
- To list the books and websites used for the study.

CHAPTER 5: CONCLUSION AND REFERENCES

5.1 Conclusion

The summer training on ML Using Python at GRAStech, Noida, gave me a strong practical base in Python programming and in the basic ideas of machine learning. In the 6 weeks of training I moved from the fundamentals of Python to the libraries Pandas, NumPy, NLTK and Scikit-learn, and used all of them in a complete project.

The Movie Recommender System successfully fulfils its objectives. It cleans and merges the TMDB 5000 datasets, represents each of the 4,806 movies as a vector of 5,000 features, calculates the cosine similarity between all the movies and recommends the five most similar movies for any selected movie. In the notebook test, the movie Gandhi gave Ramanujan, Guiana 1838, The Wind That Shakes the Barley, The Bounty and A Passage to India, which are historical dramas with a similar background and show that the system captures the theme of a movie. The model is saved in pickle files and used by an attractive Streamlit application that also shows posters, ratings and overviews from the TMDB API. The application uses caching, a timeout and automatic retries, so it keeps working on an unstable network.

Apart from technical skills, the project taught me how to plan a project, to divide it into modules, to test it at different levels and to find and remove the errors that appear in real conditions, such as network failures and mismatch between the data and the model.

5.2 System Specifications

5.2.1 H/W Requirement

Table 5.1: Hardware requirements

Component	Minimum requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended because the similarity matrix is about 185 MB)
Hard disk	At least 1 GB of free space
Network	Internet connection, required for posters and details from TMDB

5.2.2 S/W Requirement

Table 5.2: Software requirements

Component	Requirement
Operating system	Windows 10 or later (Linux and macOS also supported)
Programming language	Python 3.10 or later (3.13 was used)
Libraries	Streamlit, Pandas, NumPy, Scikit-learn, NLTK, Requests
Tools	Visual Studio Code and Jupyter Notebook
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox
External service	TMDB account and API key

5.3 Limitations of the System

- The system recommends only on the basis of the content of the movies. It does not use ratings or the watch history of users, so the recommendations are not personalized.
- The bag of words model counts words and ignores their order and meaning, so two movies with different stories but similar words may be shown as similar.
- Only the 5,000 most frequent words are used, so some rare but important words are ignored.
- All the actors of a movie are used as features; the minor actors may add noise. Using only the main actors would be more precise.
- The datasets were merged on the title, which created repeated rows because some titles are repeated. Merging on the movie ID would be more accurate.
- The dataset has only about 4,800 movies and does not contain the latest releases.
- The similarity matrix is large (about 185 MB), which increases the start-up time and the memory use.
- Posters and details need an internet connection and a valid TMDB API key.

5.4 Future Scope for Modification

- **Hybrid recommendation:** combine the content-based method with collaborative filtering using the ratings of users, to give personalized suggestions.

- **Better text features:** use TF-IDF or word embeddings instead of simple word counts, so that the meaning of the words is also considered.
- **Main cast only:** use only the first three actors and the director to reduce noise.
- **Merge on movie ID:** join the datasets on the unique ID instead of the title.
- **Larger and newer data:** use the full TMDB data or the TMDB API directly to include new releases.
- **More filters:** filter by genre, year, language and rating, and show trending and top-rated movies on the home page.
- **Trailers and streaming information:** show the trailer and the platforms where a movie can be watched.
- **User accounts and watchlist:** let users log in, rate movies and save a watchlist.
- **Smaller model:** store only the top similar movies of every movie to reduce the size and the memory use.
- **Deployment:** deploy the application on a cloud service so that anyone can use it through a link, with the API key stored as a secret.

5.5 References/Bibliography

[1] F. Ricci, L. Rokach and B. Shapira (Eds.), Recommender Systems Handbook, 2nd ed. New York, NY, USA: Springer, 2015.

[2] D. Jannach, M. Zanker, A. Felfernig and G. Friedrich, Recommender Systems: An Introduction. Cambridge, U.K.: Cambridge University Press, 2010.

[3] M. F. Porter, "An algorithm for suffix stripping," Program, vol. 14, no. 3, pp. 130-137, 1980.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conference, 2010, pp. 56-61.

[6] S. Bird, E. Klein and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O'Reilly Media, 2009.

[7] The Movie Database (TMDB), "TMDB API Documentation." [Online]. Available: https://developer.themoviedb.org/docs

[8] TMDB, "TMDB 5000 Movie Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

[9] Streamlit Inc., "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io

[10] Scikit-learn developers, "Scikit-learn User Guide." [Online]. Available: https://scikit-learn.org/stable/user_guide.html

[11] Pandas development team, "Pandas Documentation." [Online]. Available: https://pandas.pydata.org/docs

[12] NLTK Project, "Natural Language Toolkit Documentation." [Online]. Available: https://www.nltk.org

[13] Python Software Foundation, "Python 3 Documentation." [Online]. Available: https://docs.python.org/3/

[14] K. Reitz, "Requests: HTTP for Humans." [Online]. Available: https://requests.readthedocs.io

CHAPTER 6
ANNEXURES

Learning Objectives of this Chapter

- To give the menu flow, structure chart and decision table of the system.
- To give the data dictionary and the test report.
- To show sample inputs and outputs with screen snapshots.
- To give the complete coding of the project.

CHAPTER 6: ANNEXURES

A-1 Menu Flow Diagram

The application has one page, so the menu flow is a simple tree. From the home page the user reaches the search box, the select box and the button; the button opens the selected movie section and the recommended movies section.

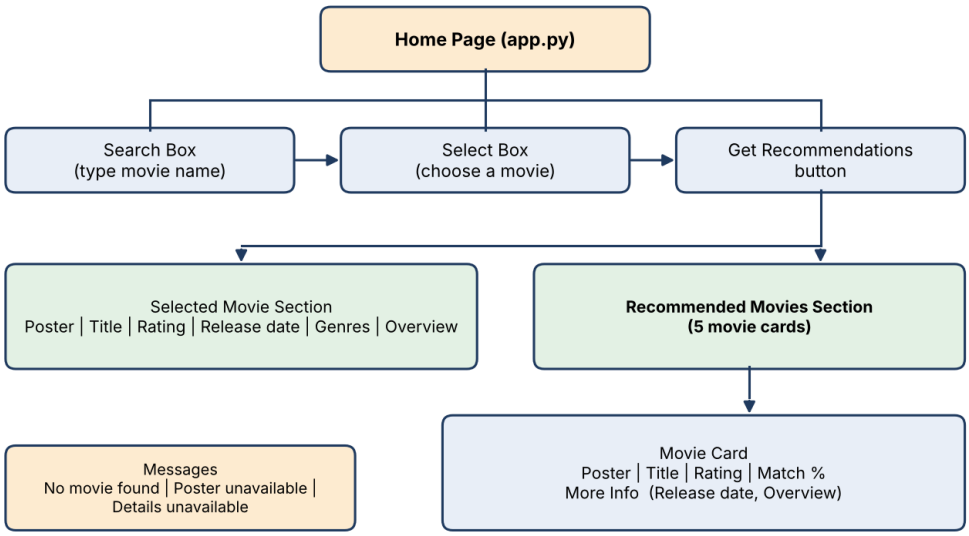


Figure 6.1: Menu flow diagram of the application

A-2 Structure Chart

The structure chart shows the functions of app.py and the functions or services that each of them uses.

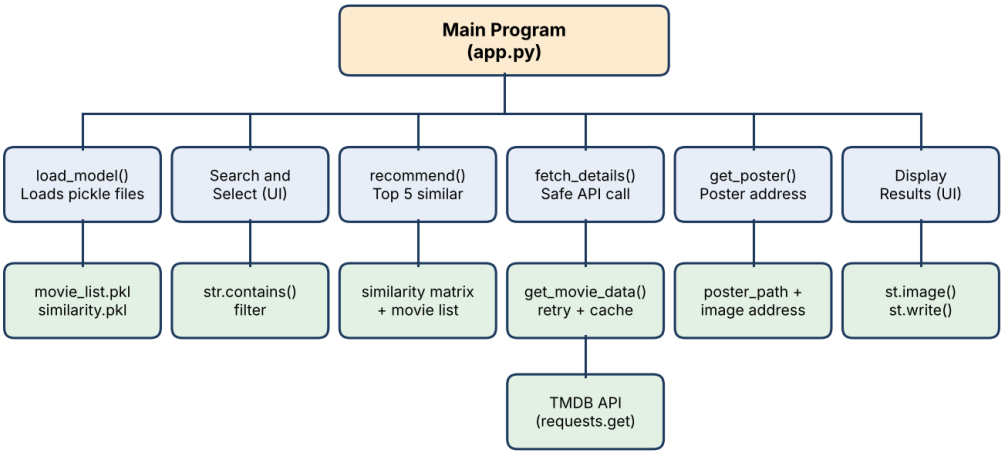


Figure 6.2: Structure chart of app.py

A-3 Decision Table/Tree

The decision table shows what the system does for the different combinations of conditions after the user opens the page.

Table 6.1: Decision table of the application

Condition / action	R1	R2	R3	R4	R5
Movies match the search text?	N	Y	Y	Y	Y
Get Recommendations clicked?	-	N	Y	Y	Y
TMDB data received?	-	-	Y	Y	N
Poster available?	-	-	Y	N	-
Show warning and stop	X				
Wait for the button		X			
Show recommendations with poster and details			X		
Show recommendations with Poster unavailable				X	
Show recommendations with Details unavailable					X

Y = yes, N = no, - = not applicable, X = action taken.

A-4 Data Dictionary

Table 6.2: Data dictionary of movie_list.pkl

Field	Data type	Description
movie_id	Integer	Unique TMDB ID of the movie (key)
title	Text	Name of the movie
tags	Text	Lower case, stemmed text made from overview, genres, keywords, cast and director

Table 6.3: Data dictionary of the model and the API data

Item	Data type	Description
vector	NumPy array (4806 x 5000)	Count of the 5,000 most frequent words for every movie
similarity	NumPy array (4806 x 4806)	Cosine similarity between every pair of movies, between 0 and 1
poster_path	Text	Last part of the poster address received from TMDB
vote_average	Decimal	Average rating of the movie on TMDB
release_date	Text	Release date in the form YYYY-MM-DD
genres	List	Genres of the movie, each with an id and a name
overview	Text	Short story of the movie

Table 6.4: Data dictionary of important variables in app.py

Variable	Description
API_KEY, BASE_URL	Key and base address of the TMDB API
movies, similarity	Movie list and similarity matrix loaded from the pickle files
search_text	Text typed by the user in the search box
movie_list	Titles of the movies that match the search text
selected_movie	Movie chosen in the select box
details	Dictionary of the TMDB response for a movie (None if it could not be fetched)
recommendations	List of five dictionaries with title, movie_id and score

A-5 Test Reports

The report below records the result of the acceptance test: every functional requirement was checked on the running application.

Table 6.5: Acceptance test report

Requirement	Checked by	Result
FR1 Load the model files at start	TC1, TC2, TC3	[Pass]
FR2 Filter movies by the text typed	TC4, TC5, TC7	[Pass]
FR3 Select one movie	TC8	[Pass]
FR4 Find five similar movies	TC8, TC9, TC15	[Pass]
FR5 Fetch details of the selected movie	TC8, TC10	[Pass]
FR6 Show poster, title, rating, match % and more info	TC10, TC11, TC12	[Pass]
FR7 Message when no movie matches	TC6	[Pass]
FR8 Work when poster or details cannot be fetched	TC13	[Pass]

Name of the tester: Kunal Upadhyay. Signature of the guide: _____ Date: _____

A-6 Sample Inputs

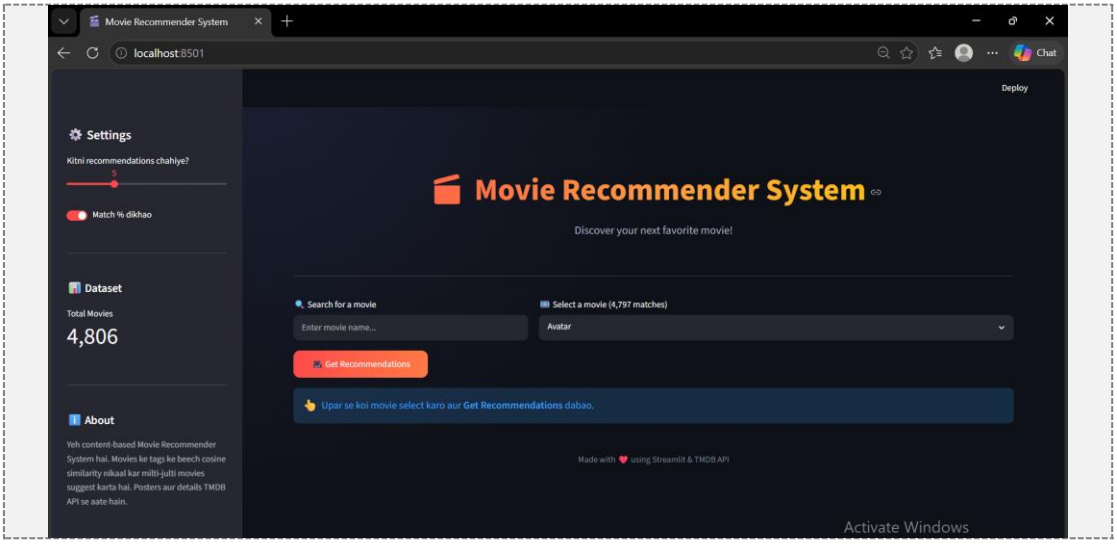


Figure 6.3: Input screen: search box and select box

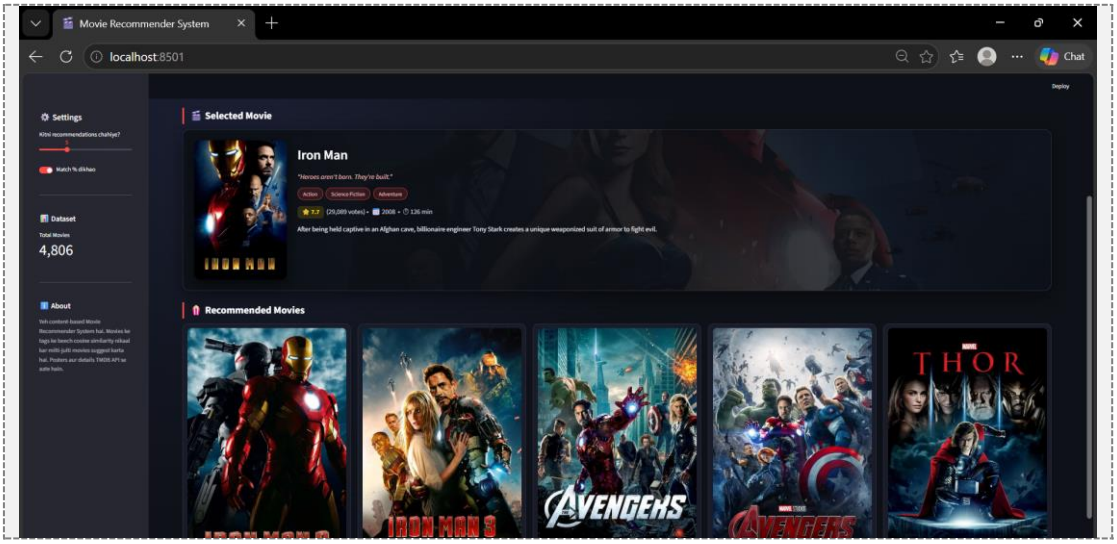


Figure 6.4: Input: search text typed and the filtered select box

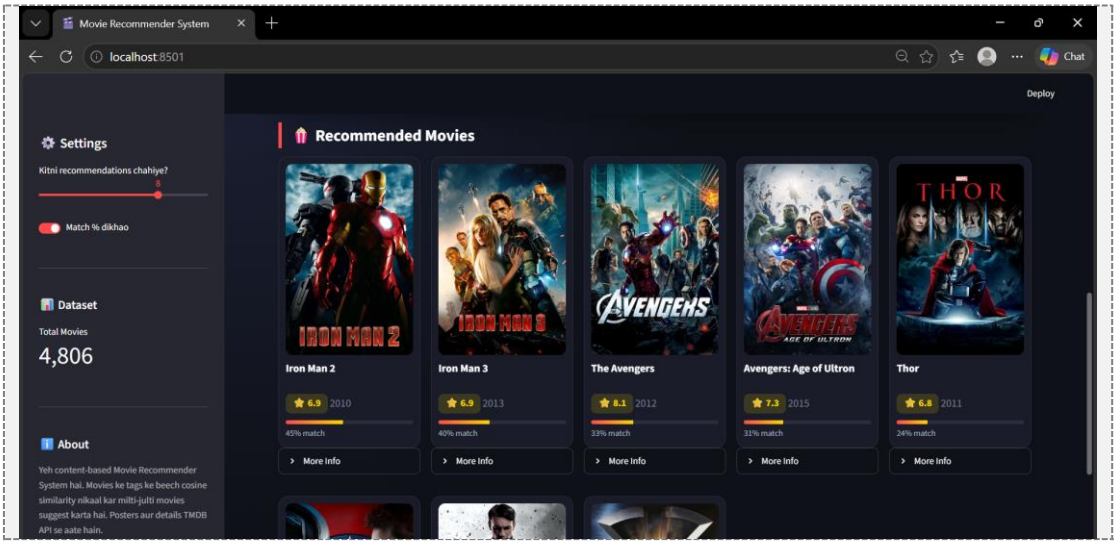


Figure 6.5: Input: Get Recommendations button

A-7 Sample Outputs

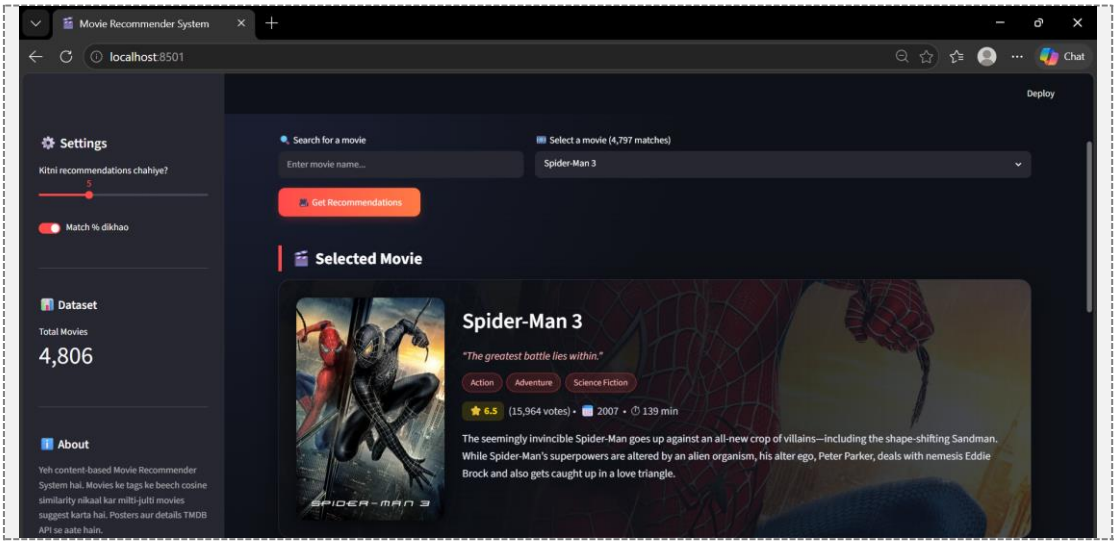


Figure 6.6: Output: selected movie section

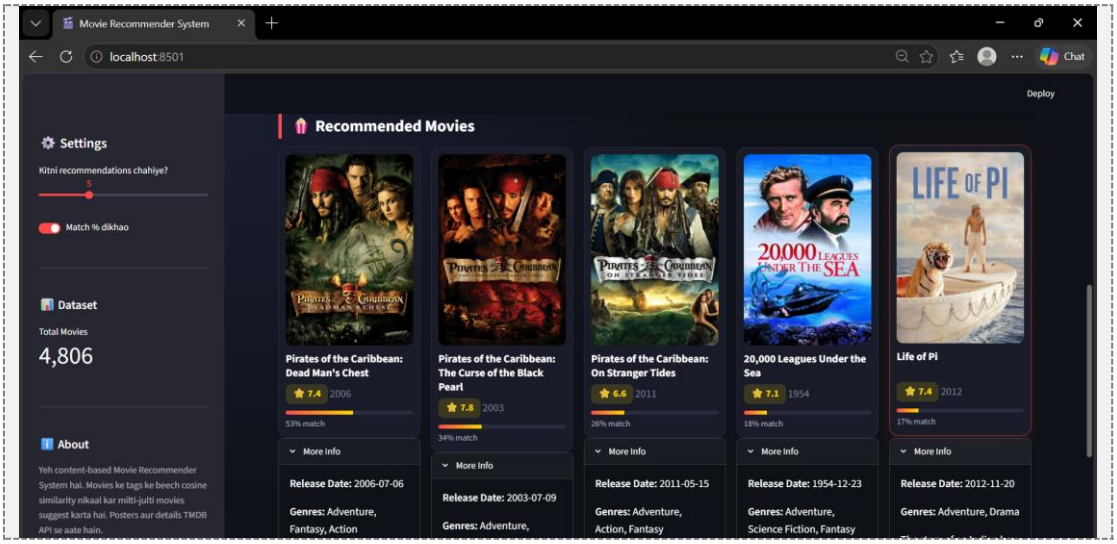


Figure 6.7: Output: five recommended movies

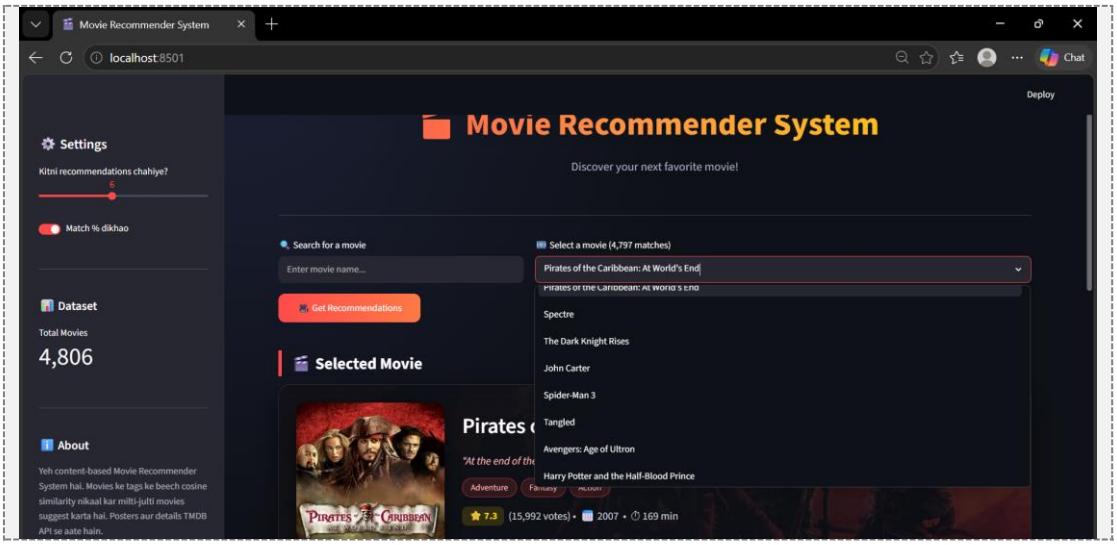


Figure 6.8: Output: More Info of a recommended movie

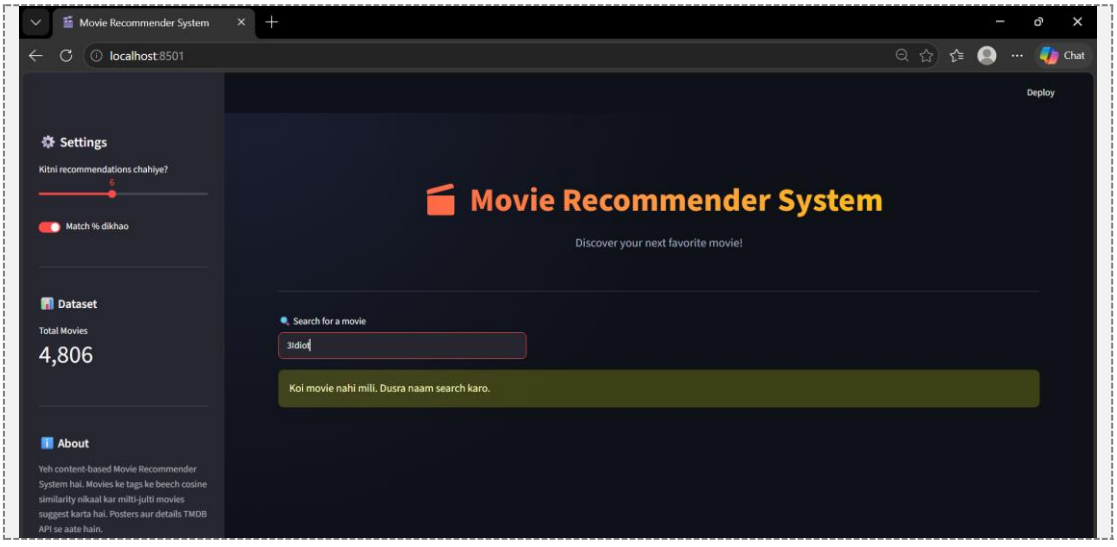


Figure 6.9: Output: warning when no movie matches

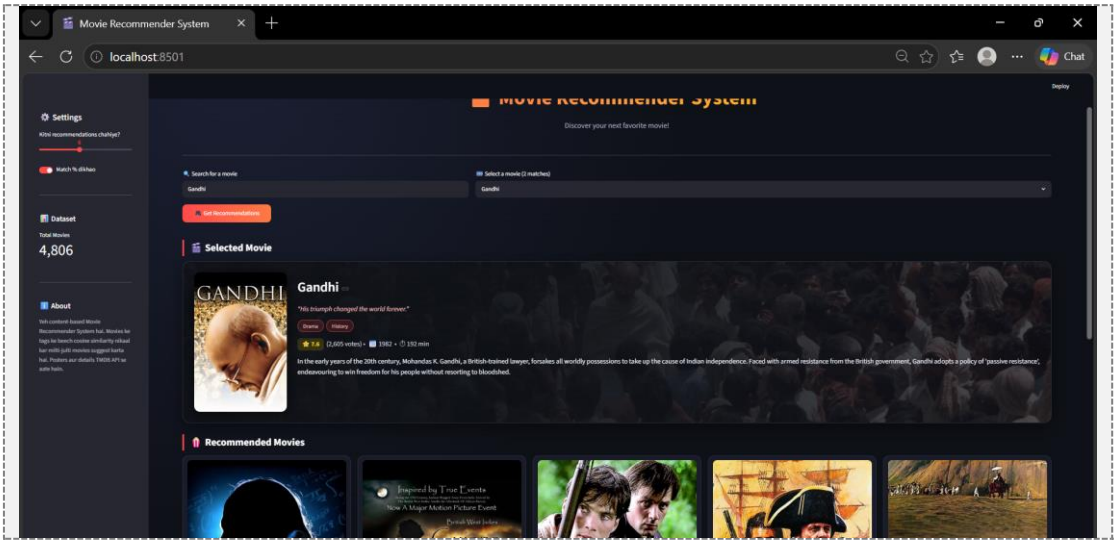


Figure 6.10: Output of the notebook: recommend('Gandhi')

A-8 Coding

The complete code of the notebook (Movie Recommender System.ipynb) and of the web application (app.py) is given below. Only the code cells of the notebook are included. Emoji characters used in the text of the application are omitted in this listing.

Code of the notebook: Movie Recommender System.ipynb

```
# ---- Cell 1 ----
import numpy as np
import pandas as pd

# ---- Cell 2 ----
import pandas as pd

movies = pd.read_csv('../Dataset/tmdb_5000_movies.csv')
credits = pd.read_csv('../Dataset/tmdb_5000_credits.csv')

# ---- Cell 3 ----
credits.sample(2)

# ---- Cell 4 ----
movies.sample(2)

# ---- Cell 5 ----
movies = movies.merge(credits,on='title')

# ---- Cell 6 ----
movies.sample(1)

# ---- Cell 7 ----
movies.info()

# ---- Cell 8 ----
movies = movies[['movie_id','title','overview','genres','keywords','cast','crew']]

# ---- Cell 9 ----
movies.head(1)

# ---- Cell 10 ----
movies.isnull().sum()
```

```
# ---- Cell 11 ----
movies.dropna(inplace=True)

# ---- Cell 12 ----
movies.duplicated().sum()

# ---- Cell 13 ----
movies.iloc[0].genres

# ---- Cell 14 ----
import ast
ast.literal_eval('[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"},
{"id": 14, "name": "Fantasy"}, {"id": 878, "name": "Science Fiction"}]')

# ---- Cell 15 ----
def convert(text):
    L = []
    for i in ast.literal_eval(text):
        L.append(i['name'])
    return L

# ---- Cell 16 ----
movies.dropna(inplace=True)

# ---- Cell 17 ----
movies['genres'] = movies['genres'].apply(convert)

# ---- Cell 18 ----
movies['keywords'] = movies['keywords'].apply(convert)
movies.head(1)

# ---- Cell 19 ----
def convert3(text):
    L = []
    counter = 0
    for i in ast.literal_eval(text):
        if counter < 3:
            L.append(i['name'])
        counter+=1
    return L

# ---- Cell 20 ----
movies['cast'] = movies['cast'].apply(convert)

# ---- Cell 21 ----
def fetch_director(text):
    L = []
    for i in ast.literal_eval(text):
        if i['job'] == 'Director':
            L.append(i['name'])
    return L

# ---- Cell 22 ----
movies['crew'] = movies['crew'].apply(fetch_director)

# ---- Cell 23 ----
movies['overview'][0]

# ---- Cell 24 ----
movies['overview'] = movies['overview'].apply(lambda x:x.split())

# ---- Cell 25 ----
movies.sample(2)

# ---- Cell 26 ----
def collapse(L):
    L1 = []
    for i in L:
        L1.append(i.replace(" ",""))
    return L1

# ---- Cell 27 ----
movies['cast'] = movies['cast'].apply(collapse)
movies['crew'] = movies['crew'].apply(collapse)
movies['genres'] = movies['genres'].apply(collapse)
```

```
movies['keywords'] = movies['keywords'].apply(collapse)

# ---- Cell 28 ----
movies.head(2)

# ---- Cell 29 ----
movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] +
movies['cast'] + movies['crew']

# ---- Cell 30 ----
new_df = movies.drop(columns=['overview','genres','keywords','cast','crew'])

# ---- Cell 31 ----
new_df['tags'] = new_df['tags'].apply(lambda x: " ".join(x))

# ---- Cell 32 ----
new_df['tags'] = new_df['tags'].apply(lambda x: x.lower())

# ---- Cell 33 ----
new_df.head(2)

# ---- Cell 34 ----
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()

# ---- Cell 35 ----
def stem(text):
    y = []
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)

# ---- Cell 36 ----
new_df['tags'] = new_df['tags'].apply(stem)

# ---- Cell 37 ----
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000,stop_words='english')

# ---- Cell 38 ----
vector = cv.fit_transform(new_df['tags']).toarray()

# ---- Cell 39 ----
vector.shape

# ---- Cell 40 ----
from sklearn.metrics.pairwise import cosine_similarity

# ---- Cell 41 ----
similarity = cosine_similarity(vector)

# ---- Cell 42 ----
new_df[new_df['title'] == 'The Lego Movie'].index[0]

# ---- Cell 43 ----
def recommend(movie):
    index = new_df[new_df['title'] == movie].index[0]
    distances = sorted(list(enumerate(similarity[index])),reverse=True,key = lambda x:
x[1])
    for i in distances[1:6]:
        print(new_df.iloc[i[0]].title)

# ---- Cell 44 ----
recommend('Gandhi')

# ---- Cell 45 ----
import pickle

# ---- Cell 46 ----
pickle.dump(new_df,open('movie_list.pkl','wb'))
pickle.dump(similarity,open('similarity.pkl','wb'))
```

Code of the web application: app.py

```
import os
import time
import pickle
import requests
import streamlit as st


st.set_page_config(
    page_title="Movie Recommender System",
    page_icon="",
    layout="wide"
)


st.markdown("""
<style>
.title {
    text-align: center;
    font-size: 3rem;
    font-weight: 800;
    background: linear-gradient(90deg, #FF4B4B, #FFD700);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
.subtitle {
    text-align: center;
    color: gray;
    margin-bottom: 20px;
}
.stButton > button {
    background-color: #FF4B4B;
    color: white;
    border-radius: 8px;
    font-weight: bold;
    border: none;
}
.stButton > button:hover {
    background-color: #E63946;
    color: white;
}
</style>
""", unsafe_allow_html=True)



BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_DIR = os.path.join(os.path.dirname(BASE_DIR), "Model")



@st.cache_resource
def load_model():
    movies = pickle.load(open(os.path.join(MODEL_DIR, "movie_list.pkl"), "rb"))
    similarity = pickle.load(open(os.path.join(MODEL_DIR, "similarity.pkl"), "rb"))
    return movies, similarity



try:
    movies, similarity = load_model()
except Exception as error:
    st.error(f"Model load nahi hua: {error}")
    st.info("Check karo ki movie_list.pkl aur similarity.pkl Model folder mein hain.")
    st.stop()

movies = movies.reset_index(drop=True)



API_KEY = "YOUR_TMDB_API_KEY_HERE"
BASE_URL = "https://api.themoviedb.org/3"



@st.cache_data(ttl=3600, show_spinner=False)
def get_movie_data(movie_id):
    for attempt in range(3):
        try:
            response = requests.get(
                f"{BASE_URL}/movie/{int(movie_id)}",
                params={"api_key": API_KEY, "language": "en-US"},
                timeout=15
            )
```

```
            response.raise_for_status()
            return response.json()
        except requests.RequestException as error:
            print(f"TMDB error (try {attempt + 1}) for movie {movie_id}: {error}")
            time.sleep(1)

    raise ConnectionError("TMDB se data nahi mil paaya")



def fetch_details(movie_id):
    try:
        return get_movie_data(movie_id)
    except Exception:
        return None



def get_poster(details):
    if details and details.get("poster_path"):
        return "https://image.tmdb.org/t/p/w500" + details["poster_path"]
    return None



def recommend(movie_title):
    movie_index = movies[movies["title"] == movie_title].index[0]

    distances = sorted(
        list(enumerate(similarity[movie_index])),
        key=lambda x: x[1],
        reverse=True
    )

    result = []
    for index, score in distances[1:6]:
        result.append({
            "title": movies.iloc[index]["title"],
            "movie_id": movies.iloc[index]["movie_id"],
            "score": score
        })
    return result



st.markdown("<div class='title'>Movie Recommender System</div>",
unsafe_allow_html=True)
st.markdown("<div class='subtitle'>Discover your next favorite movie!</div>",
unsafe_allow_html=True)

search_col, select_col = st.columns([1, 2])

with search_col:
    search_text = st.text_input("Search for a movie", placeholder="Enter movie
name...")

if search_text.strip():
    filtered = movies[movies["title"].str.contains(search_text.strip(), case=False,
na=False, regex=False)]
else:
    filtered = movies

movie_list = filtered["title"].dropna().unique().tolist()

if len(movie_list) == 0:
    st.warning("Koi movie nahi mili. Dusra naam search karo.")
    st.stop()

with select_col:
    selected_movie = st.selectbox("Select a movie", movie_list)



if st.button("Get Recommendations"):
    movie_id = movies[movies["title"] == selected_movie].iloc[0]["movie_id"]
    details = fetch_details(movie_id)

    st.subheader("Selected Movie")
    col1, col2 = st.columns([1, 3])
```

```
    with col1:
        poster = get_poster(details)
        if poster:
            st.image(poster)
        else:
            st.info("Poster available nahi hai")

    with col2:
        st.markdown(f"## {selected_movie}")
        if details:
            genres = ", ".join([g["name"] for g in details.get("genres", [])])
            st.write(f"⭐ **Rating:** {details.get('vote_average', 'N/A')}")
            st.write(f"**Release Date:** {details.get('release_date', 'N/A')}")
            st.write(f"**Genres:** {genres}")
            st.write(details.get("overview", "No overview available."))
        else:
            st.info("Details fetch nahi ho paayi. Internet ya API key check karo.")

    st.divider()

    st.subheader("Recommended Movies")

    recommendations = recommend(selected_movie)
    columns = st.columns(5)

    for column, movie in zip(columns, recommendations):
        with column:
            with st.container(border=True):
                movie_details = fetch_details(movie["movie_id"])

                poster = get_poster(movie_details)
                if poster:
                    st.image(poster)
                else:
                    st.caption("Poster unavailable")

                st.markdown(f"**{movie['title']}**")

                if movie_details:
                    st.write(f"⭐ {movie_details.get('vote_average', 'N/A')}")
                st.caption(f"Match: {round(movie['score'] * 100)}%")

                if movie_details:
                    with st.expander("More Info"):
                        st.write("**Release Date:**",
movie_details.get("release_date", "N/A"))
                        st.write(movie_details.get("overview", "No overview
available."))

st.divider()
st.caption("Made with using Streamlit & TMDB API")
```